

ENTREGA 2

PROJETO INTERDISCIPLINAR: IoT e Robótica

Professor João Trencher

Integrantes

23025389 - Leonardo Ferreira da Silva
23025405 - Maria Kassandra Alves Gomes
23025550 - Sabrinna Vicente
23025414 - Sátiro Gabriel de Souza Santos

Descrição da arquitetura final implementada

A arquitetura do sistema foi projetada com o objetivo de monitorar o consumo de alimento e água de gatos de forma automatizada, utilizando sensores e comunicação via rede.

Visão Geral da Arquitetura

O sistema é composto por três camadas principais:

1. Camada de Aquisição de Dados (Hardware)
2. Camada de Processamento (Microcontrolador)
3. Camada de Armazenamento e Visualização (Software)

Camada de Aquisição de Dados

Essa camada é responsável por coletar informações do ambiente físico.

Componentes utilizados:

- Sensor de peso (para medir consumo de ração)
- Sensor de nível de água (conectado ao pino analógico A0)
- Sistema de identificação por RFID

Estes sensores permitem captar:

- quantidade de alimento consumido
- nível de água disponível
- identificação individual do animal

Camada de Processamento

O processamento dos dados é realizado por um **NodeMCU ESP8266**.

Funções principais:

- leitura dos sensores
- filtragem de ruído nas medições
- identificação do gato via RFID
- cálculo de consumo (diferença de peso e nível)
- organização dos dados para envio

Além disso, o sistema aplica algoritmos simples de otimização, como:

- média de leituras para reduzir erro
- descarte de valores inconsistentes

Camada de Armazenamento e Visualização

Após o processamento, os dados são enviados via Wi-Fi para uma aplicação externa (backend).

Funções dessa camada:

- armazenamento dos dados históricos
- organização das informações por animal
- possibilidade de análise futura
- suporte à visualização dos dados.

Fluxo de Funcionamento do Sistema

O funcionamento do sistema ocorre da seguinte forma:

1. O sensor RFID identifica a presença de um gato
2. O sistema inicia o monitoramento
3. Sensores de peso e nível coletam dados continuamente
4. Os dados são processados e filtrados
5. O consumo é calculado
6. As informações são enviadas para armazenamento

Ajustes em relação à proposta inicial

Durante o desenvolvimento, algumas adaptações foram necessárias:

- **Remoção do monitoramento da caixa de areia**, para simplificar a implementação
- **Foco maior em alimentação e hidratação**, por serem dados mais diretos e relevantes
- Simplificação da lógica embarcada, concentrando o processamento principal no microcontrolador

Evidências

```
1 import 'package:firebase_auth/firebase_auth.dart';
2
3 class AuthService {
4   final FirebaseAuth _auth = FirebaseAuth.instance;
5
6   // Stream para monitorar o estado da autenticação
7   Stream<User?> get user => _auth.authStateChanges();
8
9   // Login com e-mail e senha
10  Future<UserCredential?> login(String email, String password) async {
11    try {
12      return await _auth.signInWithEmailAndPassword(
13        email: email,
14        password: password,
15      );
16    } catch (e) {
17      rethrow;
18    }
19  }
20
21  // Cadastro com e-mail e senha
22  Future<UserCredential?> register(String email, String password) async {
23    try {
24      return await _auth.createUserWithEmailAndPassword(
25        email: email,
26        password: password,
27      );
28    } catch (e) {
29      rethrow;
30    }
31  }
32
33  // Logout
34  Future<void> logout() async {
35    await _auth.signOut();
36  }
37
38  // Password Reset
39  Future<void> sendPasswordResetEmail(String email) async {
40    try {
41      await _auth.sendPasswordResetEmail(email: email);
42    } catch (e) {
43      rethrow;
44    }
45  }
46
47  // Delete Account
48  Future<void> deleteAccount() async {
```

Entrega 2 > Backend > cache_service.dart

```
import 'dart:convert';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:smart_feeder/models/feeder_data.dart';

class CacheService {
  static const String _feederDataKey = 'cached_feeder_data';
  static const String _lastUserEmailKey = 'last_user_email'; // "Cookie" simulation
  static const String _themeModeKey = 'theme_mode';

  final SharedPreferences _prefs;

  CacheService(this._prefs);

  // --- Feeder Data Cache ---

  Future<void> cacheFeederData(FeederData data) async {
    final String jsonString = jsonEncode(data.toMap());
    await _prefs.setString(_feederDataKey, jsonString);
  }

  FeederData? getCachedFeederData() {
    final String? jsonString = _prefs.getString(_feederDataKey);
    if (jsonString != null) {
      try {
        final Map<String, dynamic> map = jsonDecode(jsonString);
        return FeederData.fromMap(map);
      } catch (e) {
        return null;
      }
    }
    return null;
  }

  // --- "Cookies" (Persistent UI state) ---

  Future<void> saveLastUserEmail(String email) async {
    await _prefs.setString(_lastUserEmailKey, email);
  }

  String? getLastUserEmail() {
    return _prefs.getString(_lastUserEmailKey);
  }

  Future<void> saveThemeMode(String mode) async {
    await _prefs.setString(_themeModeKey, mode);
  }

  String? getThemeMode() {
```

```

28     return null;
29   }
30 }
31 return null;
32 }
33
34 // --- "Cookies" (Persistent UI state) ---
35
36 Future<void> saveLastUserEmail(String email) async {
37   await _prefs.setString(_lastUserEmailKey, email);
38 }
39
40 String? getLastUserEmail() {
41   return _prefs.getString(_lastUserEmailKey);
42 }
43
44 Future<void> saveThemeMode(String mode) async {
45   await _prefs.setString(_themeModeKey, mode);
46 }
47
48 String? getThemeMode() {
49   return _prefs.getString(_themeModeKey);
50 }
51
52 Future<void> clearCache() async {
53   await _prefs.remove(_feederDataKey);
54 }
55 }
56

```

auth_service.dart cache_service.dart feeder_service.dart X

src > Entrega 2 > Backend > feeder_service.dart

```

1  import "dart:async";
2  import "package:smart_feeder/models/feeder_data.dart";
3
4  abstract class FeederService {
5    Stream<FeederData> get feederDataStream;
6    Future<void> triggerManualFeeding();
7    Future<void> tareScale();
8  }
9
10

```

src > Entrega 2 > Backend > history_service.dart

```
1 import 'package:cloud_firestore/cloud_firestore.dart';
2 import 'package:smart_feeder/models/feeding_event.dart';
3
4 class HistoryService {
5   final FirebaseFirestore _firestore = FirebaseFirestore.instance;
6
7   Future<void> logEvent(FeedingEvent event) async {
8     await _firestore.collection('feeding_history').add(event.toMap());
9   }
10
11   Stream<List<FeedingEvent>> getHistoryStream() {
12     return _firestore
13       .collection('feeding_history')
14       .orderBy('timestamp', descending: true)
15       .limit(50)
16       .snapshots()
17       .map((snapshot) {
18         return snapshot.docs.map((doc) {
19           return FeedingEvent.fromMap(doc.data(), doc.id);
20         }).toList();
21       });
22   }
23
24   Future<List<FeedingEvent>> getEventsByPet(String rfidTag) async {
25     final snapshot = await _firestore
26       .collection('feeding_history')
27       .where('rfidTag', isEqualTo: rfidTag)
28       .orderBy('timestamp', descending: true)
29       .get();
30
31     return snapshot.docs.map((doc) {
32       return FeedingEvent.fromMap(doc.data(), doc.id);
33     }).toList();
34   }
35 }
36
```

auth_service.dart

cache_service.dart

feeder_service.dart

history_service.dart

src > Entrega 2 > Backend > mock_feeder_service.dart

```
1 import "dart:async";
2 import "package:flutter/foundation.dart";
3 import "package:smart_feeder/models/feeder_data.dart";
4 import "package:smart_feeder/services/feeder_service.dart";
5
6 class MockFeederService implements FeederService {
7   final _controller = StreamController<FeederData>.broadcast();
8   FeederData _lastData = FeederData(
9     waterLevel: 45.0,
10    waterStatus: "WATER_DETECTED",
11    foodWeight: 150.0,
12    lastPetDetected: "Rex (RFID: 12345)",
13    isOnline: true,
14  );
15
16  MockFeederService() {
17    Timer.periodic(const Duration(seconds: 5), (timer) {
18      _lastData = FeederData(
19        waterLevel: 45.0,
20        waterStatus: "WATER_DETECTED",
21        foodWeight: _lastData.foodWeight + (timer.tick % 2 == 0 ? 5 : -5),
22        lastPetDetected: timer.tick % 3 == 0 ? "None" : "Rex (RFID: 12345)",
23        isOnline: true,
24      );
25      _controller.add(_lastData);
26    });
27  }
28
29  @override
30  Stream<FeederData> get feederDataStream => _controller.stream;
31
32  @override
33  Future<void> triggerManualFeeding() async {
34    debugPrint("Service: Sending feeding command (MOCK)...");
35  }
36
37  @override
38  Future<void> tareScale() async {
39    debugPrint("Service: Sending TARE command (MOCK)...");
40  }
41 }
42
```

Entrega 2 > Backend > mqtt_feeder_service.dart

```
1 import "dart:async";
2 import "dart:convert";
3 import "package:flutter/foundation.dart";
4 import "package:mqtt_client/mqtt_client.dart";
5 import "package:smart_feeder/core/constants/app_constants.dart";
6 import "package:smart_feeder/models/feeder_data.dart";
7 import "package:smart_feeder/services/feeder_service.dart";
8 import "package:smart_feeder/services/mqtt_setup/mqtt_setup.dart";
9
10 class MqttFeederService implements FeederService {
11   final MqttClient _client;
12   final _controller = StreamController<FeederData>.broadcast();
13   Completer<void> _connectionCompleter = Completer<void>();
14
15   MqttFeederService()
16     : _client = createMqttClient(
17         AppConstants.mqttBroker,
18         "flutter_client_${DateTime.now().millisecondsSinceEpoch}",
19       ) {
20     _init();
21   }
22
23   Future<void> _init() async {
24     _client.onDisconnected = _onDisconnected;
25     _client.onConnected = _onConnected;
26
27     final connMess = MqttConnectMessage()
28       .withClientIdentifier(_client.clientIdentifier)
29       .startClean();
30     _client.connectionMessage = connMess;
31
32     try {
33       debugPrint("MQTT Client: Connecting to ${AppConstants.mqttBroker}...");
34       await _client.connect();
35
36       _client.updates?.listen((List<MqttReceivedMessage<MqttMessage?>>? messages) {
37         if (messages == null) return;
38
39         final recMess = messages[0].payload as MqttPublishMessage;
40         final payload = MqttPublishPayload.bytesToStringAsString(recMess.payload.message);
41
42         if (messages[0].topic == AppConstants.topicStatus) {
43           _handleIncomingData(payload);
44         }
45       });
46     } catch (e) {
47       debugPrint("MQTT Client: Connection error - $e");
48       _client.disconnect();
49       if (!connectionCompleter.isCompleted) {
```

Ln 1, Col 1 - Save


```

    } catch (e) {
        if (!_connectionCompleter.isCompleted) {
            _connectionCompleter.completeError(e);
        }
    }
}

void _handleIncomingData(String payload) {
    try {
        final Map<String, dynamic> data = jsonDecode(payload);
        _controller.add(FeederData.fromMap(data));
    } catch (e) {
        debugPrint("Error decoding MQTT JSON: $e");
    }
}

void _onConnected() {
    debugPrint("MQTT Client: CONNECTED!");
    _client.subscribe(AppConstants.topicStatus, MqttQos.atMostOnce);
    if (!_connectionCompleter.isCompleted) {
        _connectionCompleter.complete();
    }
}

void _onDisconnected() {
    debugPrint("MQTT Client: DISCONNECTED");
    if (_connectionCompleter.isCompleted) {
        _connectionCompleter = Completer<void>();
    }
}

@override
Stream<FeederData> get feederDataStream => _controller.stream;

Future<void> _ensureConnected() async {
    if (_client.connectionStatus?.state == MqttConnectionState.connected) return;

    if (_client.connectionStatus?.state == MqttConnectionState.connecting) {
        await _connectionCompleter.future;
        return;
    }

    _connectionCompleter = Completer<void>();
    await _client.connect();
    await _connectionCompleter.future;
}

```

```

        await _connectionCompleter.future,
    }

    @override
    Future<void> triggerManualFeeding() async {
        try {
            await _ensureConnected();
            final builder = MqttClientPayloadBuilder();
            builder.addString("FEED");
            _client.publishMessage(
                AppConstants.topicCommand,
                MqttQos.atLeastOnce,
                builder.payload!,
            );
            debugPrint("MQTT Client: FEED command sent.");
        } catch (e) {
            debugPrint("Error triggering feeding: $e");
        }
    }

    @override
    Future<void> tareScale() async {
        try {
            await _ensureConnected();
            final builder = MqttClientPayloadBuilder();
            builder.addString("TARE");
            _client.publishMessage(
                AppConstants.topicCommand,
                MqttQos.atLeastOnce,
                builder.payload!,
            );
            debugPrint("MQTT Client: TARE command sent.");
        } catch (e) {
            debugPrint("Error taring scale: $e");
        }
    }
}

```

```
cache_service.dart  feeder_service.dart  history_service.dart  mock_feeder_service.dart  n
c > Entrega 2 > Backend > network_config_service.dart
1  import 'package:http/http.dart' as http;
2  import 'dart:developer' as dev;
3
4  class NetworkConfigService {
5      static const String espIp = '192.168.4.1';
6      static const String configEndpoint = 'http://$espIp/config';
7
8      Future<bool> configureWifi(String ssid, String password) async {
9          dev.log('Iniciando configuração de rede...', name: 'NetworkService');
10         dev.log('SSID: $ssid', name: 'NetworkService');
11         dev.log('Endpoint: $configEndpoint', name: 'NetworkService');
12
13         try {
14             final response = await http.post(
15                 Uri.parse(configEndpoint),
16                 headers: {
17                     'Content-Type': 'application/x-www-form-urlencoded',
18                 },
19                 body: {
20                     'ssid': ssid,
21                     'pass': password,
22                 },
23             ).timeout(const Duration(seconds: 15));
24
25             dev.log('Resposta recebida: ${response.statusCode}', name: 'NetworkService');
26             dev.log('Corpo da resposta: ${response.body}', name: 'NetworkService');
27
28             return response.statusCode == 200;
29         } catch (e) {
30             dev.log('ERRO NA REQUISIÇÃO: $e', name: 'NetworkService', error: e);
31             return false;
32         }
33     }
34 }
35
```

dart

feeder_service.dart

history_service.dart

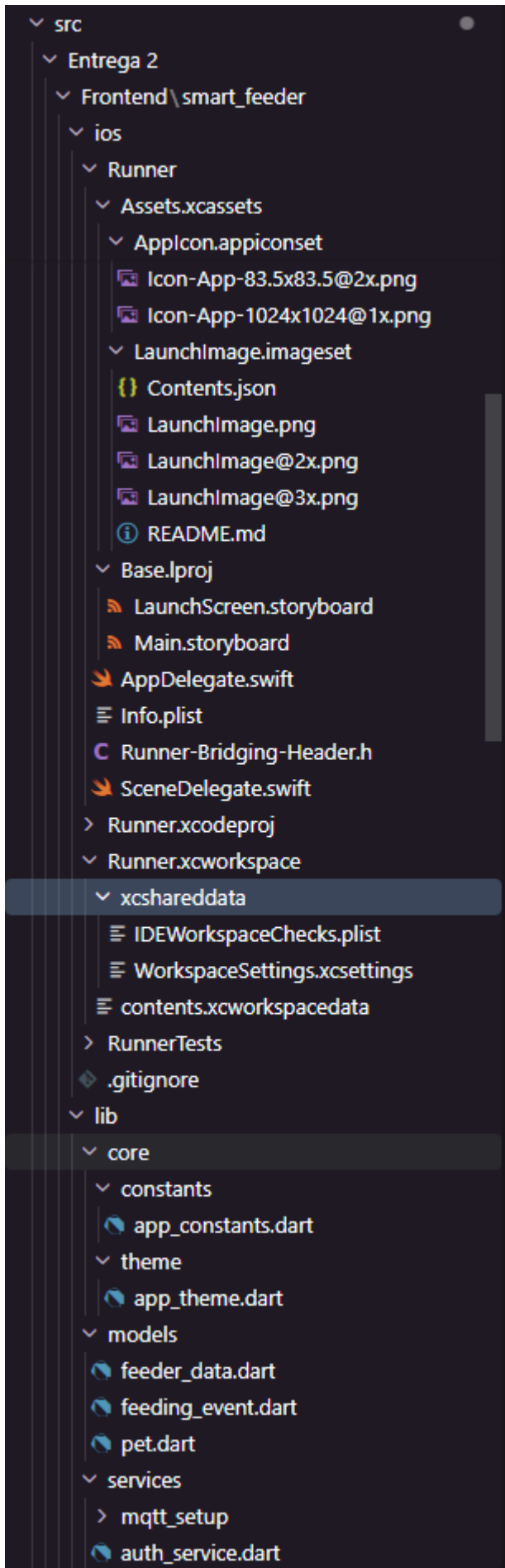
mock_feeder_service.dart

mqtt_feeder_service.dart

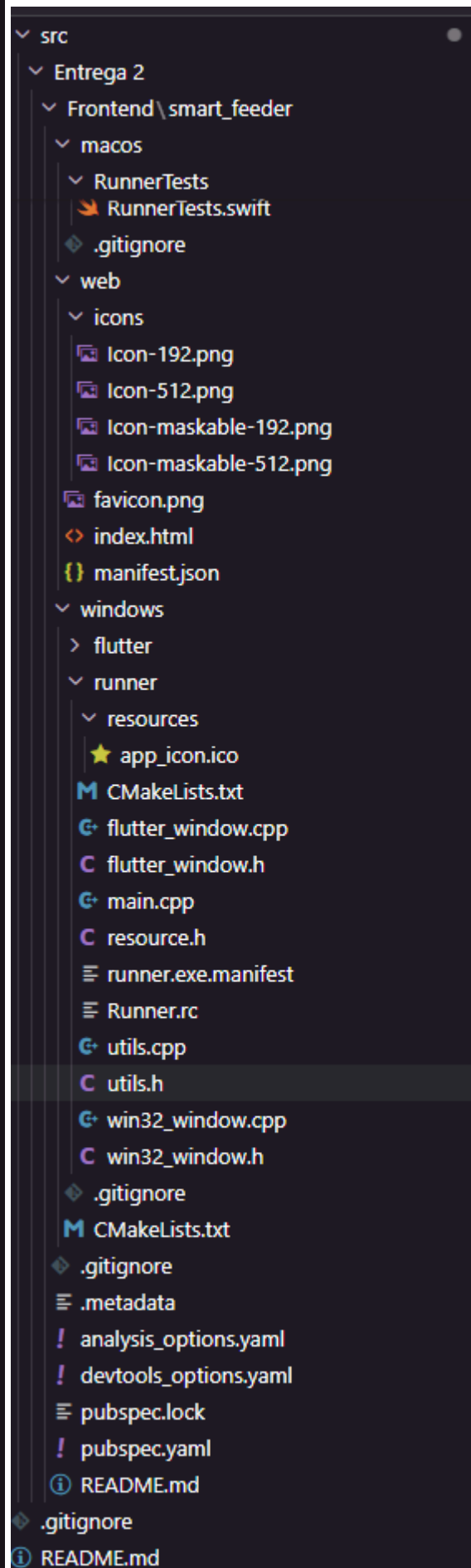
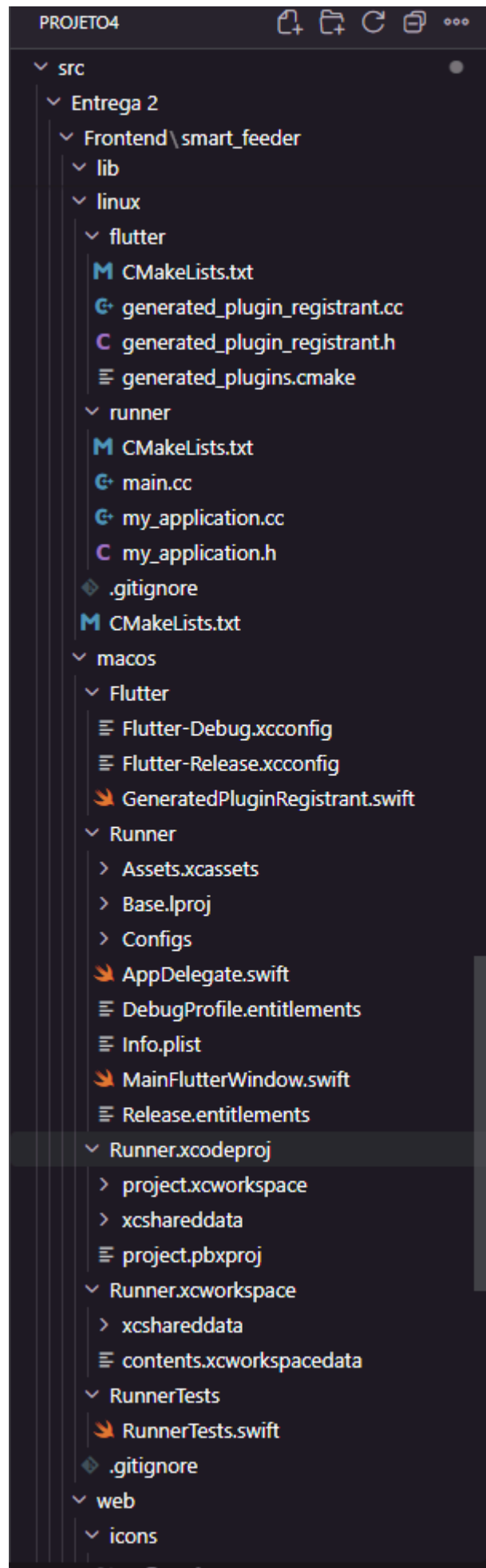
src > Entrega 2 > Backend > pet_service.dart

```
1 import 'package:flutter/foundation.dart';
2 import 'package:cloud_firestore/cloud_firestore.dart';
3 import 'package:smart_feeder/models/pet.dart';
4
5 class PetService {
6   final FirebaseFirestore _firestore = FirebaseFirestore.instance;
7
8   Future<Pet?> getPetByRfid(String rfidTag) async {
9     try {
10       final doc = await _firestore.collection('pets').doc(rfidTag).get();
11       if (doc.exists && doc.data() != null) {
12         return Pet.fromMap(doc.data());
13       }
14     } catch (e) {
15       debugPrint("PetService: Error getting pet by RFID - $e");
16     }
17     return null;
18   }
19
20   Future<void> registerPet(Pet pet) async {
21     try {
22       await _firestore.collection('pets').doc(pet.rfidTag).set(pet.toMap());
23     } catch (e) {
24       debugPrint("PetService: Error registering pet - $e");
25       rethrow;
26     }
27   }
28
29   Future<List<Pet>> getAllPets() async {
30     try {
31       final snapshot = await _firestore.collection('pets').get();
32       return snapshot.docs.map((doc) => Pet.fromMap(doc.data())).toList();
33     } catch (e) {
34       debugPrint("PetService: Error getting all pets - $e");
35       return [];
36     }
37   }
38 }
39
```

- Frontend\smart_feeder
 - android
 - app
 - src
 - debug
 - AndroidManifest.xml
 - main
 - profile
 - build.gradle.kts
 - gradle\wrapper
 - gradle-wrapper.properties
 - .gitignore
 - build.gradle.kts
 - gradle.properties
 - settings.gradle.kts
 - ios
 - Flutter
 - AppFrameworkInfo.plist
 - Debug.xcconfig
 - Release.xcconfig
 - Runner
 - Assets.xcassets
 - AppIcon.appiconset
 - Contents.json
 - Icon-App-20x20@1x.png
 - Icon-App-20x20@2x.png
 - Icon-App-20x20@3x.png
 - Icon-App-29x29@1x.png
 - Icon-App-29x29@2x.png
 - Icon-App-29x29@3x.png
 - Icon-App-40x40@1x.png
 - Icon-App-40x40@2x.png
 - Icon-App-40x40@3x.png
 - Icon-App-60x60@2x.png
 - Icon-App-60x60@3x.png
 - Icon-App-76x76@1x.png
 - Icon-App-76x76@2x.png
 - Icon-App-83.5x83.5@2x.png
 - Icon-App-1024x1024@1x.png



- src
 - Entrega 2
 - Frontend\smart_feeder
 - lib
 - models
 - feeder_data.dart
 - feeding_event.dart
 - pet.dart
 - services
 - mqtt_setup
 - auth_service.dart
 - cache_service.dart
 - feeder_service.dart
 - history_service.dart
 - mock_feeder_service.dart
 - mqtt_feeder_service.dart
 - network_config_service.dart
 - pet_service.dart
 - utils
 - seeder.dart
 - view_models
 - auth_view_model.dart
 - feeder_view_model.dart
 - history_view_model.dart
 - network_config_view_model.dart
 - theme_view_model.dart
 - views
 - dashboard_view.dart
 - delete_account_view.dart
 - forgot_password_view.dart
 - history_view.dart
 - login_view.dart
 - network_config_view.dart
 - register_view.dart
 - settings_view.dart
 - widgets
 - app_drawer.dart
 - status_card.dart
 - firebase_options.dart
 - main.dart
 - linux
 - flutter
 - CMakeLists.txt





FEEDING HISTORY

GLOBAL LOG

PET INSIGHTS

TODAY FOOD

164g

TODAY WATER

111%

TODAY



LUKE

19:04

5.0%



LUCAS

19:02

2.0%



LUKE

18:56

74.0g



LUCAS

18:45

2.0%



LUCAS

18:35

85.6%



LUCAS

18:32

1.0%



REX

18:31

15.0%

19:54

5G 55



FEEDING HISTORY

GLOBAL LOG

PET INSIGHTS



LUCAS

RFID: 4311900D

WEEKLY FOOD TREND

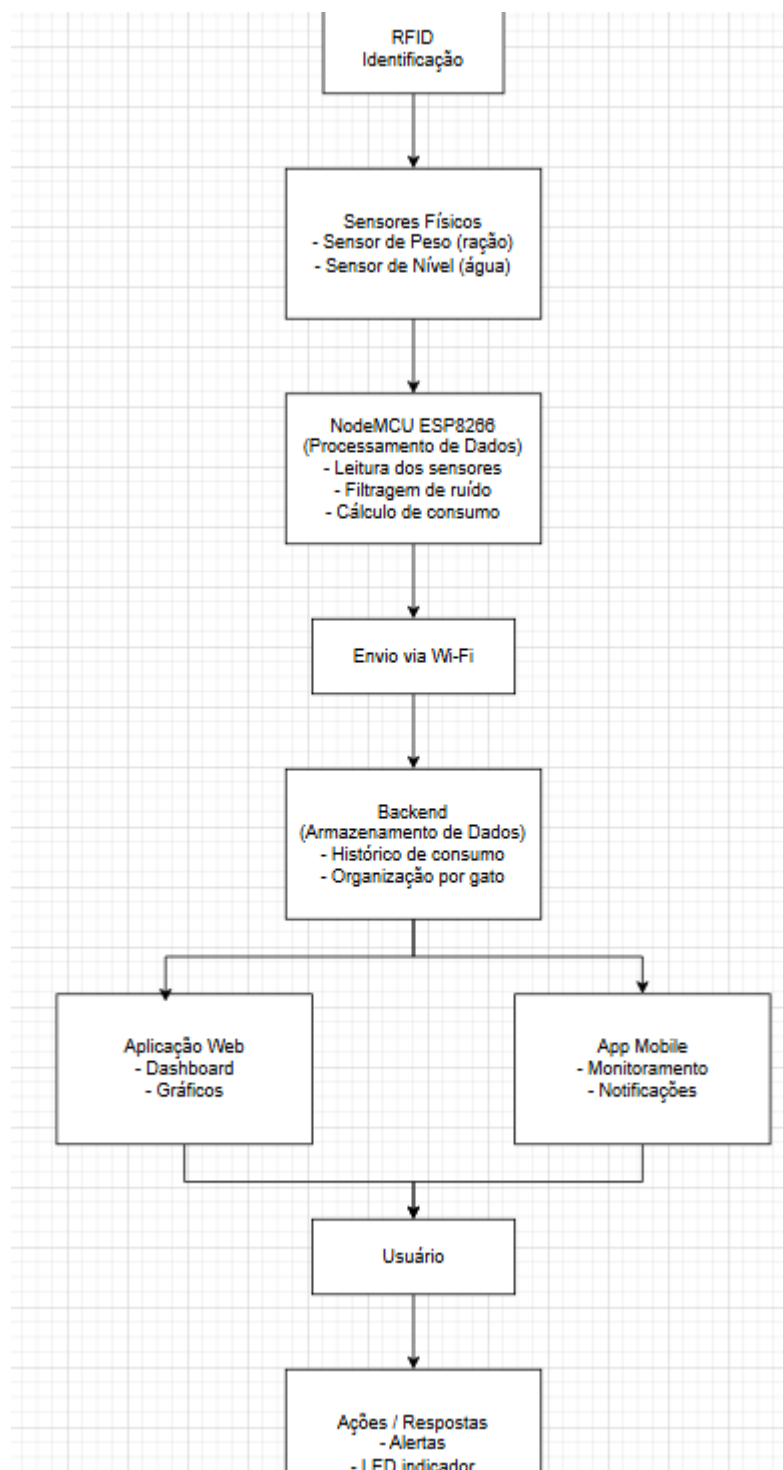


BEHAVIOR ANALYSIS

Pet is currently WAITING.... Data indicates consistent patterns.



Demonstração do fluxo de dados e ações



O fluxo do sistema inicia na coleta de dados pelos sensores físicos responsáveis pelo monitoramento do consumo de água e ração. Em seguida, o NodeMCU ESP8266 realiza o processamento das informações, incluindo filtragem de ruídos e organização dos dados.

Após o processamento, as informações são enviadas via Wi-Fi para o backend, onde são armazenadas e organizadas por animal identificado via RFID. Posteriormente, os dados ficam disponíveis para visualização nas aplicações web e mobile, permitindo acompanhamento remoto e geração de alertas ao usuário.

Registro dos testes realizados e validação dos requisitos

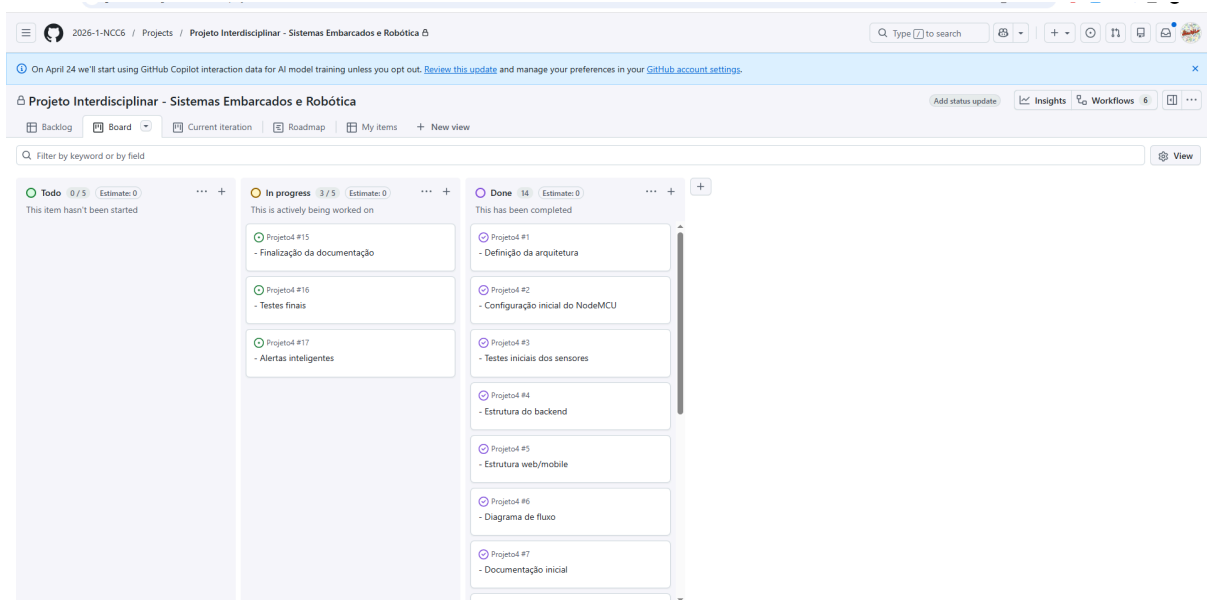
Teste	Requisito	Entrada	Saída Esperada	Resultado
T1	Leitura do sensor de água	Nível baixo no reservatório	Valor abaixo do limite definido	Sucesso
T2	Leitura do sensor de peso	Alteração no peso da ração	Registro da variação de peso	Sucesso
T3	Identificação RFID	Aproximação da tag	Identificação do gato	Sucesso
T4	Envio de dados	Dados processados pelo NodeMCU	Dados recebidos no backend	Sucesso
T5	Visualização no sistema	Dados armazenados	Exibição no app/web	Sucesso

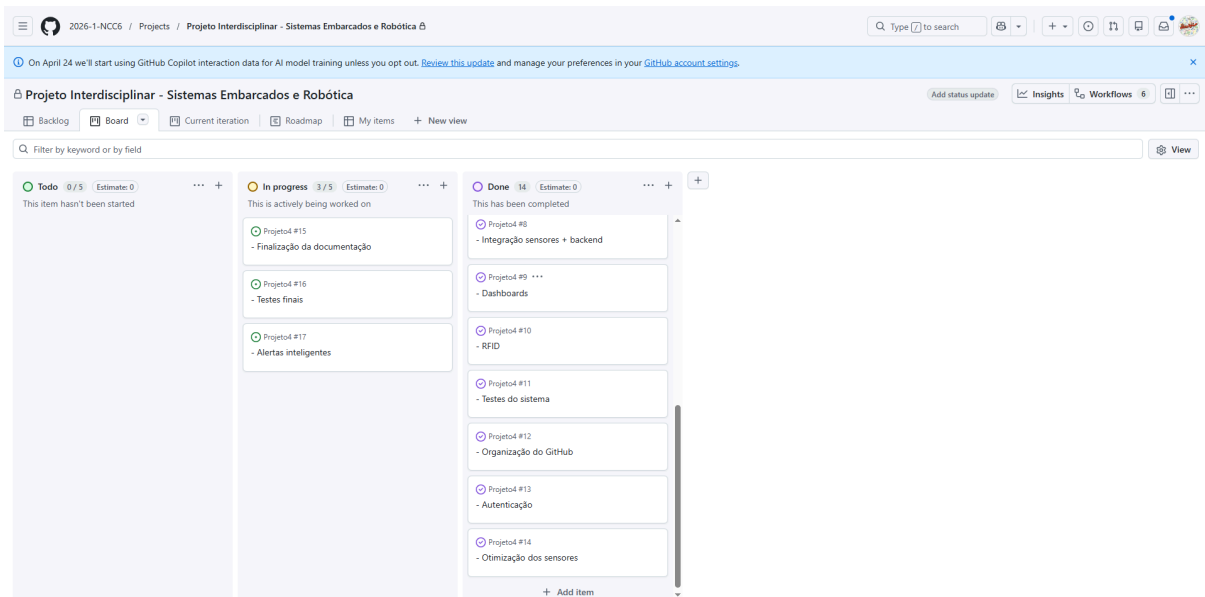
Foram realizados testes funcionais para validar os requisitos definidos no sistema. Os testes envolveram a leitura dos sensores de nível e peso, a identificação por RFID e o envio de dados ao backend. Os resultados demonstraram que o sistema foi capaz de capturar, processar e transmitir as informações corretamente. As evidências obtidas incluem registros no monitor serial, bem como a visualização dos dados nas aplicações desenvolvidas. Dessa forma, conclui-se que os requisitos propostos foram atendidos de maneira satisfatória.

Backlog

O gerenciamento das tarefas do projeto foi realizado utilizando ferramentas de controle de backlog e acompanhamento das atividades da equipe. O backlog atualizado encontra-se disponível no repositório do projeto:

<https://github.com/orgs/2026-1-NCC6/projects/3/views/2>





Análise crítica das limitações do sistema e propostas de melhoria

Apesar de o sistema proposto atender ao objetivo principal de monitorar o consumo de ração e água dos gatos, ainda apresenta algumas limitações naturais por se tratar de uma solução em desenvolvimento e em fase de prototipagem. Uma das principais limitações está relacionada à confiabilidade das medições dos sensores, especialmente em ambientes reais, onde podem ocorrer ruídos, variações de leitura e necessidade de calibração frequente. Além disso, a comunicação do sistema depende da estabilidade da rede Wi-Fi, o que pode comprometer o envio contínuo dos dados em caso de falhas de conexão.

Outra limitação importante é a precisão da identificação por RFID, que pode sofrer interferência de distância, posicionamento da tag ou leitura incorreta em cenários com múltiplos animais. Também é necessário considerar que a integração entre hardware, backend, aplicação web e aplicativo mobile ainda exige ajustes para garantir maior robustez, sincronização e confiabilidade no fluxo completo de dados. Em termos de segurança, o sistema também pode evoluir na proteção das credenciais, na validação das mensagens enviadas e no controle de acesso às informações dos usuários.

Como proposta de melhoria, o grupo pretende aperfeiçoar a calibração dos sensores e aplicar filtros de leitura para reduzir ruídos e valores fora do padrão. Também será importante implementar um mecanismo de armazenamento temporário dos dados em caso de queda de internet, evitando perda de informações. No campo da identificação, melhorias na posição do leitor RFID e no desenho físico do protótipo podem aumentar a taxa de leitura correta.

Do ponto de vista de software, uma evolução relevante seria ampliar as funcionalidades do backend para gerar alertas inteligentes com base em padrões de consumo, além de melhorar os dashboards do site e do aplicativo mobile para facilitar a interpretação dos dados pelo usuário. Em versões futuras, o sistema também pode incorporar previsões de

comportamento, notificações automáticas e maior integração com recursos de segurança e monitoramento em tempo real.

De forma geral, o projeto já demonstra viabilidade técnica e potencial de uso prático, mas ainda pode ser aprimorado em precisão, estabilidade, segurança e experiência de uso. Essas melhorias tornam o sistema mais confiável e aproximam a solução de um produto final aplicável em cenários reais.

Conclui-se que o sistema apresentou funcionamento satisfatório para os objetivos definidos nesta etapa do projeto, demonstrando viabilidade técnica da solução proposta e potencial para futuras evoluções.